

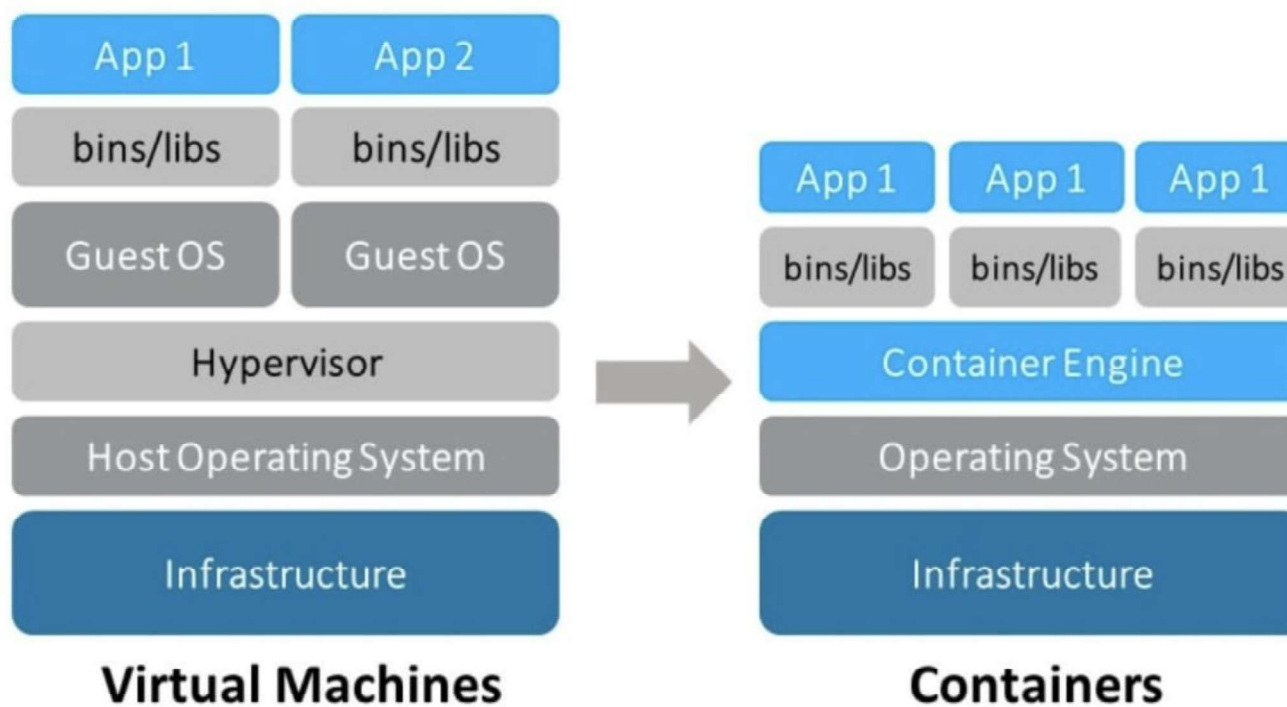
Docker - part 1

Basic concepts

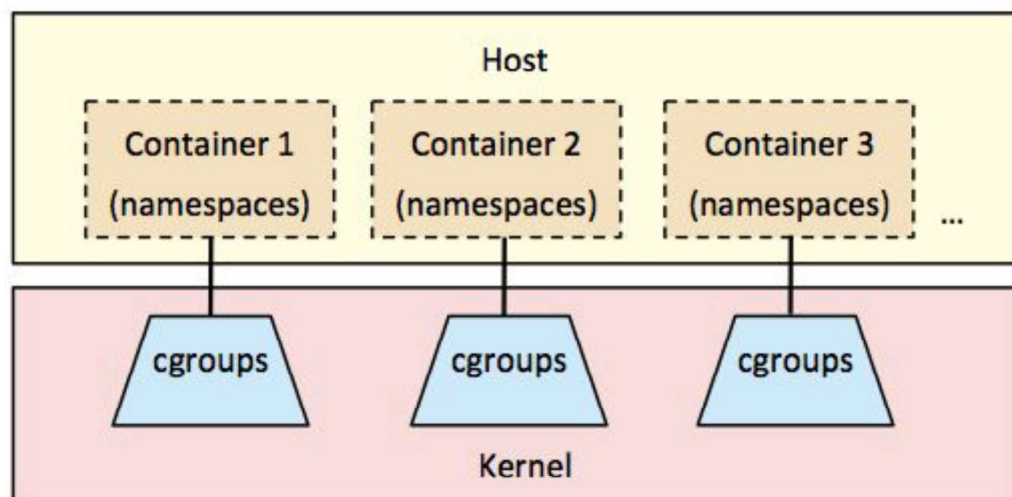
Marica Antonacci (INFN Bari)
marica.antonacci@ba.infn.it



Virtual machines vs Containers



Containers = combination of namespaces & cgroups

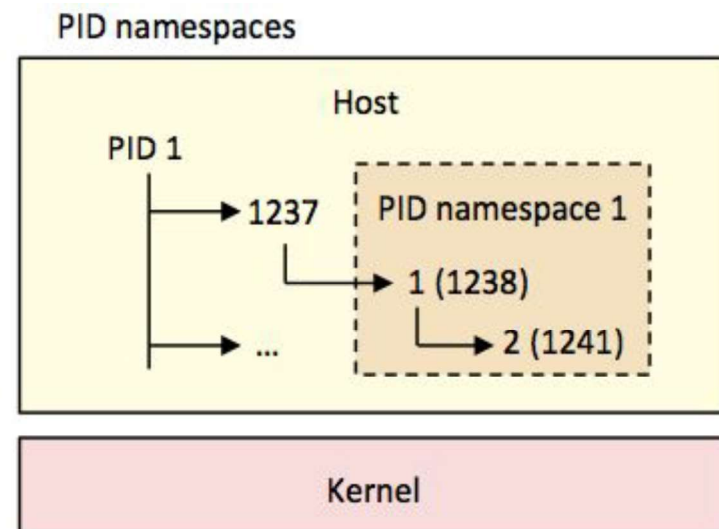


Restricting visibility: Namespaces

Linux namespaces: It is a feature of Linux kernel to isolate resources from each other. This allows one set of Linux processes to see one group of resources while allowing another set of Linux processes to see a different group of resources.

There are several kinds of namespaces in Linux: Mount (mnt), Process ID (PID), Network (net), User ID (user), Control group (cgroup), and Interprocess Communication (IPC).

For example, two processes in two different mounted namespaces may have different views of what the mounted root file system is. Each container can be associated with a specific set of namespaces, and these namespaces are used inside these containers only.

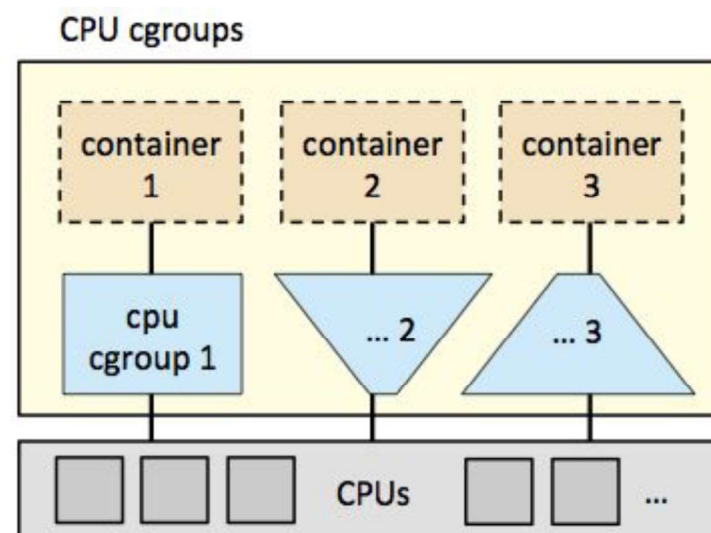


Restricting usage: Control groups

cgroups provide an effective mechanism for **resource limitation**.

With cgroups, you can control and manage system resources (CPU, Memory, Networking, ...) per Linux process, increasing overall resource utilization efficiency.

Cgroups allow to control resource utilization per container.



Docker main components

- **Docker containers:** Isolated user-space environments running the same or different applications and sharing the same host OS. Containers are created from Docker images.
- **Docker images:** Docker templates that include application libraries and applications. Images are used to create containers and you can bring up containers immediately. You can create and update your own custom images as well as download build images from Docker's public registry.
- **Docker registries:** This is an images store. Docker registries can be public or private, meaning that you can work with images available over the internet or create your own registry for internal purposes. One popular public Docker registry is [Docker Hub](https://hub.docker.com/).

What is a docker image?

Images are **multi-layered self-contained files** that act as the **template** for creating containers. They are like a frozen, read-only copy of a container.

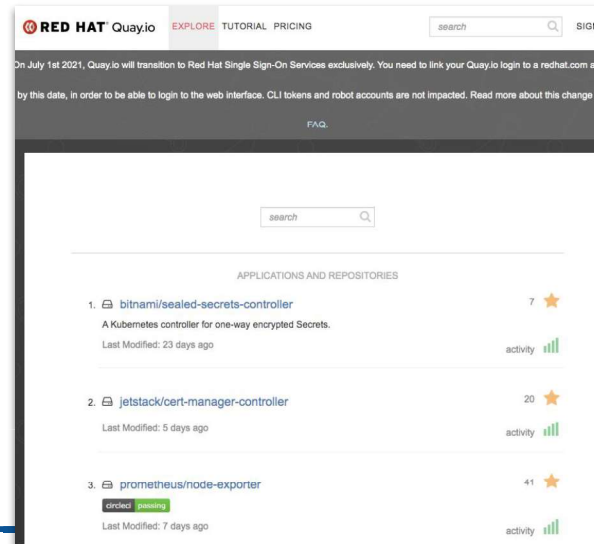
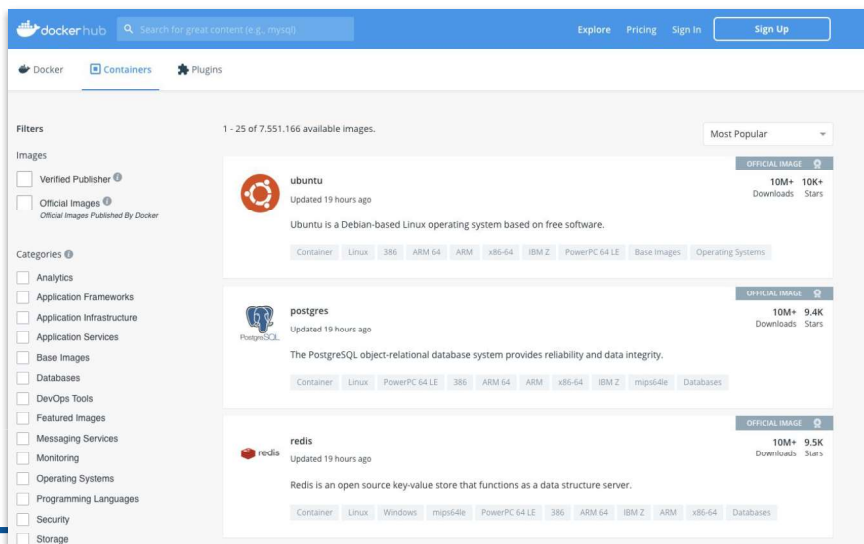
In the past, different container engines had different image formats. But later on, the Open Container Initiative (OCI) defined a standard specification for container images which is complied by the major containerization engines out there. This means that an image built with Docker can be used with another runtime like Podman.

Images can be exchanged through registries.

Docker registry

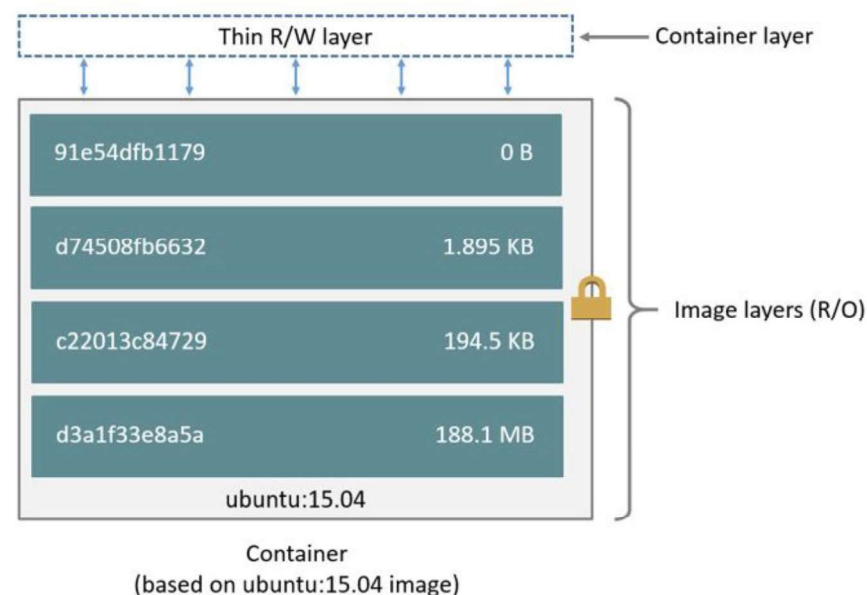
An image registry is a centralized place where you can upload your images and can also download images created by others.

Docker Hub is the default public registry for Docker. Another very popular image registry is **Quay** by Red Hat.



Docker image layers

- A Docker Image consists of read-only layers built on top of each other.
- Docker uses the **Union File System** (UFS) to build an image.
- The image is shared across containers.
- Each time Docker launches a container from an image, it adds a thin writable layer, known as the container layer, which stores all changes to the container throughout its runtime.

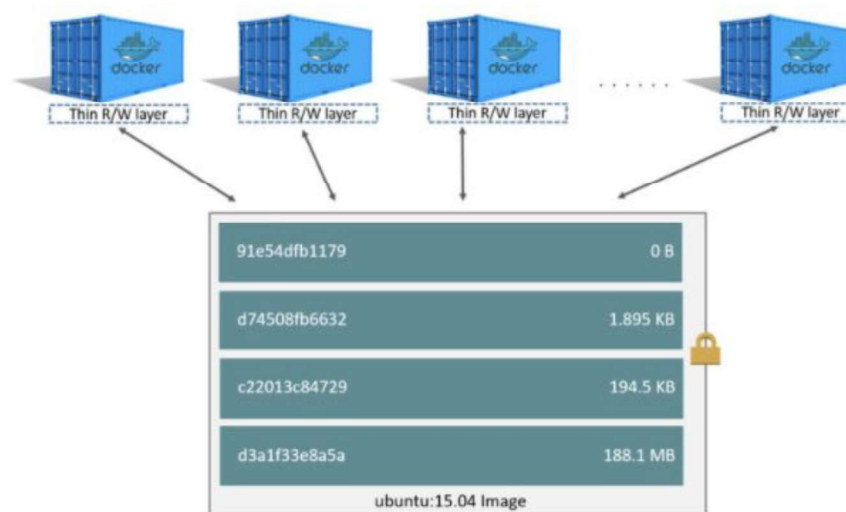


Docker image vs container

Each container has its own writable container layer, and all changes are stored in this container layer.

Multiple containers can share access to the same underlying image and yet have their own data state.

When the container is deleted, the writable layer is also deleted. The underlying image remains unchanged.



Copy-On-Write mechanism

COW is a standard UNIX pattern that provides a single shared copy of some data until the data is modified.

Docker makes use of copy-on-write technology with both images and containers. This **CoW strategy optimizes both image disk space usage and the performance of container start times**. At start time, Docker only has to create the thin writable layer for each container.

Containers that write a lot of data consume more space than containers that do not. This is because most write operations consume new space in the container's thin writable top layer.

Note: for write-heavy applications, you should not store the data in the container. Instead, use Docker volumes, which are independent of the running container and are designed to be efficient for I/O. In addition, volumes can be shared among containers and do not increase the size of your container's writable layer. (Source: [Docker docs](#))